量化自主交易Agent

设计方案 V2.0

——基于OpenClaw的智能量化交易系统

运行平台	管理资金	策略体系	数据来源
OpenClaw	150,000元	三策略四阶段	量化评分系统

V2.0 更新内容
1. 集成OpenClaw超级智能体平台 2. 更新标的配置策略（基于最新实战经验） 3. 优化各阶段ETF选择逻辑

2026年3月

一、OpenClaw平台集成架构

1.1 OpenClaw平台简介

OpenClaw（原名Clawdbot → Moltbot）是一个开源的超级AI智能体平台，由蚂蚁集团主导开发。其核心特点是"本地优先"的AI Agent运行时环境，深度整合大模型能力与本地系统工具。平台具备以下核心能力：本地系统权限执行Shell命令、私有记忆存储、跨应用协作、多步骤任务自动规划、工具调用与API集成、浏览器自动化操作。这些能力使其成为量化交易Agent的理想运行平台。

1.2 为什么选择OpenClaw

能力维度	OpenClaw支持	量化交易需求	匹配度
Shell命令执行	□ 完整支持	运行量化评分脚本	完美匹配
Python脚本调用	□ 完整支持	执行phase_identifier.py	完美匹配
数据库连接	□ 支持	读取量化评分数据	完美匹配
多步骤规划	□ 核心能力	阶段识别→策略选择→交易执行	完美匹配
记忆系统	□ 私有存储	持仓记录、历史决策	完美匹配
本地优先	□ 核心设计	数据安全、隐私保护	完美匹配

表1：OpenClaw与量化交易需求匹配分析

1.3 系统集成架构

以下是QuantAutoTrader与OpenClaw平台的集成架构图示。整个系统分为四层：数据层、分析层、决策层、执行层。OpenClaw作为核心运行时环境，协调各层组件完成自主交易决策。

```
┌──────────────
```

二、标的配置策略（V2.0更新）

核心发现：基于最新实战经验总结，不同市场阶段应选择不同类型的指数标的，这对策略收益有显著影响。以下为更新后的标的配置策略：

2.1 抄底反弹阶段：中证系列优先

核心原则：在市场极度恐慌时，优先选择跌幅较大的中证系列指数（中证500、中证1000、中证2000），避免选择特定市场指数（科创50、创业50等），因为后市表现差异很大，特定市场指数的反弹力度和确定性不如宽基中证系列。

优先级	ETF代码	名称	配置比例	选择原因
★★★	510500	中证500ETF	40%	跌幅大、反弹确定性高
★★★	512100	中证1000ETF	35%	修复力度适中、流动性好
★★☆	159531	中证2000ETF	25%	小盘股弹性大

表2：抄底反弹阶段标的配置

避免标的：科创50、创业50、科创100、创业200等特定市场指数。原因：这些指数波动太大，容易被情绪主导，后市表现差异很大，抄底风险高。

2.2 震荡阶段：中证系列为主

核心原则：震荡阶段市场轮动太快，应优先配置中证500、中证1000、中证2000。这些宽基指数在震荡市场中表现更稳定，不容易因为某个行业的快速轮动而受损。避免配置特定市场指数或行业指数。

优先级	ETF代码	名称	配置比例	选择原因
★★★	510500	中证500ETF	40%	中盘代表，震荡中表现稳定
★★★	512100	中证1000ETF	35%	小盘代表，轮动适应性强
★★☆	159531	中证2000ETF	25%	小微盘，高弹性

表3：震荡阶段标的配置

2.3 主升浪阶段：创业/科创系列优先

核心原则：主升浪意味着市场有主线强势行业带领，应选择强势的市场指数（科创50、创业50等），甚至可以配置3成以下的主线行业指数。这与抄底反弹阶段完全相反，需要灵活切换思路。

优先级	ETF代码	名称	配置比例	选择原因
★★★	588000	科创50ETF	25%	科创龙头，主升浪先锋
★★★	159949	创业50ETF	25%	创业龙头，高成长性
★★☆	510500	中证500ETF	25%	基础配置，稳定性
★★☆	主线行业ETF	半导体/AI/新能源	≤25%	跟随主线，超额收益

表4：主升浪阶段标的配置

主线行业选择原则：选择与主升浪周期共振的行业、优先国家政策支持的核心赛道、关注成交量放大的强势板块、避免滞涨和轮动结束的板块。主升浪要"追涨"，重仓主线板块获取超额收益！

三、详细Skill定义（OpenClaw实现版）

3.1 Skill-1：市场阶段识别

Skill描述（写入OpenClaw的Skill定义）：

```
名称：market_phase_identifier 描述：基于量化评分数据识别当前市场阶段，输出阶段类型、置信度、操作建议
输入数据源： - MySQL数据库：market_status_history表（量化评分数据） - 查询语句：SELECT trade_time,
composite_score, daily_score, min120_score, min60_score, min30_score, average_rise, up_ratio FROM
market_status_history WHERE trade_time >= DATE_SUB(CURDATE(), INTERVAL 60 DAY) AND TIME(trade_time) =
'15:00:00' ORDER BY trade_time ASC 处理步骤： 1. 获取最近60天的量化评分数据 2.
调用phase_identifier.py执行阶段识别 3. 输出当前阶段、子阶段、置信度、操作建议 输出格式（JSON）： {
"current_phase": "阶段1：主升上涨浪-确认期", "main_phase": "主升上涨浪", "sub_phase": "确认期",
"confidence": 95, "action": "加仓至7-9成", "position_ratio": 0.80, "characteristics": ["综合评分9.1分",
"突破8.8分确认"] } 调用命令： python /path/to/phase_identifier.py
```

3.2 Skill-2：策略信号生成与标的匹配

这是V2.0新增的核心Skill，将阶段识别结果转换为具体交易信号，并根据新经验总结匹配最优标的配置。

```
名称：strategy_signal_generator 描述：根据市场阶段生成交易信号，并匹配最优标的配置 输入： - current_phase:
当前市场阶段（来自Skill-1） - composite_score: 综合评分 - multi_period_scores: 多周期评分 -
current_holdings: 当前持仓信息 处理逻辑（决策树）： IF 主升上涨浪（阶段1）: IF 首次突破8.8分: signal =
"买入" position = 70-80% targets = [科创50ETF:25%, 创业50ETF:25%, 中证500ETF:25%, 主线行业ETF:≤25%] ELIF
已持仓 AND 评分>=7.5: signal = "持有" ELIF 连续2天跌破7.5: signal = "卖出" reason = "趋势结束" ELIF
调整杀跌浪（阶段3）: IF 评分<=1.2 AND 多周期<=2 AND 跌幅<-1.5%: signal = "梯度买入" day1_position = 2成
day2_position = 3-4成 day3_position = 5-6成 targets = [中证500ETF:40%, 中证1000ETF:35%, 中证2000ETF:25%] #
注意：避免科创50、创业50！ ELIF 评分反弹至6-7: signal = "卖出" reason = "反弹到位" ELIF 震荡浪（阶段2或4）:
IF 评分触及区间下沿: signal = "买入" position = 20-30% targets = [中证500ETF:40%, 中证1000ETF:35%,
中证2000ETF:25%] ELIF 评分触及区间上沿: signal = "卖出" reason = "震荡高点" 输出格式（JSON）： { "signal":
"买入/卖出/持有/观望", "position_target": 0.75, "targets": [ {"code": "588000", "name": "科创50ETF",
"ratio": 0.25, "amount": 37500}, {"code": "159949", "name": "创业50ETF", "ratio": 0.25, "amount": 37500},
... ], "reason": "主升浪确认，突破8.8分", "risk_level": "低" }
```

3.3 Skill-3：仓位动态配置

```
名称：position_allocator 描述：根据交易信号和资金约束，计算具体买卖数量 输入： - signal:
交易信号（来自Skill-2） - targets: 目标标的配置 - current_holdings: 当前持仓 - available_cash: 可用资金 -
constraints: 约束条件 约束条件： - 总仓位上限：80%（12万） - 单笔上限：33%（5万） - 单一标的：40%（6万） -
止损线：10%（1.5万强制清仓） - 预警线：5%（0.75万降低仓位） 处理逻辑： 1. 检查约束条件是否满足 2.
计算各标的需要买卖的数量 3. 生成订单列表 输出格式（JSON）： { "orders": [ { "code": "588000", "action":
"buy", "shares": 1000, "estimated_price": 1.05, "estimated_amount": 10500, "reason": "主升浪配置" } ],
"total_amount": 105000, "remaining_cash": 45000, "position_ratio": 0.70 }
```

3.4 Skill-4：交易执行引擎

```
名称：trade_executor 描述：通过券商API执行买卖订单 输入： - orders: 订单列表（来自Skill-3） 执行流程： 1.
连接券商交易接口 2. 遍历订单列表 3. 执行买卖操作 4. 等待成交确认 5. 更新持仓记录 风控检查（执行前）： -
订单金额是否超过单笔上限 - 买入后总仓位是否超过80% - 可用资金是否充足 - 标的是否在白名单内
输出格式（JSON）： { "executed_orders": [ { "code": "588000", "action": "buy", "shares": 1000, "price":
1.048, "amount": 10480, "time": "2026-03-10 09:25:03", "status": "success" } ], "failed_orders": [],
"portfolio_update": { "total_value": 105480, "cash": 44520, "position_ratio": 0.703 } }
```

3.5 Skill-5：风险监控预警

```
名称：risk_monitor 描述：实时监控账户风险指标，触发预警 监控指标： -
账户收益率：(当前净值-初始资金)/初始资金 - 当前仓位：持仓市值/总资金 - 最大回撤：历史最高净值后的最大跌幅 -
策略胜率：盈利次数/总交易次数 - 日均收益：30日累计收益/30 预警规则： | 指标 | 正常范围 | 预警阈值 |
触发动作 | |------|----------|----------|----------| | 账户收益率 | >0% | <-5% | 降低仓位至50% | | 当前仓位
| 0-80% | >75% | 禁止新买入 | | 最大回撤 | <5% | >8% | 发送告警 | | 账户亏损 | <5% | ≥10% | 强制清仓 |
输出格式（JSON）： { "status": "warning", "alerts": [ { "type": "drawdown_warning", "message":
"账户亏损达到5.2%，接近预警线", "suggested_action": "建议降低仓位至50%" } ], "metrics": { "return_rate":
-0.052, "position_ratio": 0.65, "max_drawdown": 0.068 } }
```

3.6 Skill-6：交易日志分析

```
名称：trade_logger 描述：记录交易决策过程，生成分析报告 记录内容： -
每次决策的完整上下文（评分、阶段、信号） - 交易执行的详细记录（价格、数量、时间） - 持仓变化和收益统计 -
系统错误和异常日志 报告生成： - 日报：当日决策摘要、持仓变动 - 周报：本周交易统计、策略表现 -
月报：月度收益分析、优化建议 存储位置： - 本地文件：/data/trading_logs/ - 格式：JSON + Markdown
```

四、每日执行流程（OpenClaw编排）

4.1 时间触发规则

时间	触发动作	Skill链	产出
15:00	数据更新	数据获取脚本	最新量化评分
15:05	阶段识别	Skill-1	市场阶段判断
15:10	策略决策	Skill-2 → Skill-3	交易信号+标的
15:15	风险检查	Skill-5	风险报告
次日09:25	交易执行	Skill-4	成交记录
交易后	日志记录	Skill-6	决策日志

表5：每日执行流程

4.2 OpenClaw编排脚本示例

```
# OpenClaw Agent编排脚本示例 # 文件：quant_trader_workflow.yaml name: quant_auto_trader description:
量化自主交易Agent工作流 version: "2.0" triggers: - type: cron schedule: "0 15 * * 1-5" # 工作日15:00触发
timezone: "Asia/Shanghai" skills: - name: market_phase_identifier type: shell command: python
/app/scripts/phase_identifier.py timeout: 60s - name: strategy_signal_generator type: llm prompt_template:
| 你是量化交易策略决策器。根据以下信息生成交易信号： 当前阶段：{phase_result} 综合评分：{composite_score}
当前持仓：{current_holdings} 请严格按照策略规则输出交易决策。 output_format: json - name:
position_allocator type: python script: /app/scripts/position_allocator.py - name: trade_executor type: api
endpoint: ${BROKER_API_URL}/order method: POST - name: risk_monitor type: python script:
/app/scripts/risk_monitor.py schedule: "*/5 * * * *" # 每5分钟检查 - name: trade_logger type: python
script: /app/scripts/trade_logger.py workflow: steps: - skill: market_phase_identifier outputs:
[phase_result] - skill: strategy_signal_generator inputs: phase_result: ${{ steps[0].outputs.phase_result
}} composite_score: ${{ context.composite_score }} current_holdings: ${{ context.current_holdings }}
outputs: [signal, targets] - skill: position_allocator inputs: signal: ${{ steps[1].outputs.signal }}
targets: ${{ steps[1].outputs.targets }} outputs: [orders] - skill: trade_executor inputs: orders: ${{
steps[2].outputs.orders }} outputs: [execution_result] - skill: trade_logger inputs: all_steps: ${{ steps
}} constraints: max_position: 0.80 max_single_trade: 50000 stop_loss: 0.10 alert_threshold: 0.05
```

五、风险控制机制（15万资金版）

5.1 资金约束（硬性规则）

约束项	规则	金额	触发动作
总仓位上限	持仓市值≤80%	≤120,000元	禁止买入
单笔上限	单笔≤33%	≤50,000元	拆分订单
单一标的	单标的≤40%	≤60,000元	超量禁止
止损线	总亏损≥10%	≥15,000元	强制清仓
预警线	总亏损≥5%	≥7,500元	降仓至50%

表6：资金约束规则

5.2 策略级止损规则

策略	最大仓位	止损条件	时间止损
策略一：抄底反弹	50%	单策略亏损5%	持仓>20天
策略二：主升浪做多	80%	连续2天跌破7.5分	持仓>30天
策略三：箱体震荡	30%	单次亏损3%	持仓>10天

表7：策略级止损规则

5.3 人工干预机制

虽然Agent设计为自主运行，但必须保留以下人工干预权限，确保在极端情况下可以及时止损：

干预类型	触发方式	权限范围
紧急停止	手动按钮/命令	立即停止所有交易，保持当前持仓
强制清仓	手动按钮/命令	卖出所有持仓，转为现金
策略暂停	手动设置	暂停某个策略的信号生成
仓位调整	手动设置	临时调整总仓位上限
标的白名单	手动设置	增减可交易标的范围

表8：人工干预机制

六、实施路径与关键里程碑

6.1 四阶段实施计划

阶段	时间	任务	验收标准	资金
阶段一 系统搭建	1-2周	部署OpenClaw、接入数据源、配置交易API	数据连通、API可用	0
阶段二 模拟运行	2-4周	模拟环境运行、验证决策逻辑	20日无异常	0
阶段三 小资金实盘	1-2月	5万实盘运行、监控表现	收益为正	5万
阶段四 正式运行	持续	扩大至15万、正式自主运行	持续监控优化	15万

表9：四阶段实施计划

6.2 关键成功因素

1. 相信系统：历史回测已证明策略有效性（策略一胜率85.7%，策略二胜率100%），不要因为短期波动而改变系统决策。OpenClaw的记忆系统会记录所有决策历史，保持一致性执行。

2. 严格执行：OpenClaw的优势在于消除人为情绪干扰。确保Agent严格按照Skill定义执行，不要随意覆盖系统信号。只在极端情况下使用人工干预权限。

3. 标的选择差异化：牢记新经验总结——抄底反弹和震荡阶段选择中证系列，主升浪阶段选择科创/创业系列。这是提高收益的关键优化点。

4. 持续优化：利用OpenClaw的"边用边训"能力（AReaL框架），定期分析Agent表现，优化决策参数。每月生成优化报告，持续改进策略。

6.3 监控与报告

报告类型	频率	内容	交付方式
日报	每日收盘后	决策摘要、持仓变动、收益情况	微信/邮件
周报	每周五	交易统计、策略表现、风险指标	邮件
月报	月末	收益分析、策略评估、优化建议	PDF报告
异常报告	即时	异常类型、影响、处理措施	微信+短信

表10：监控报告体系

附录：Agent技术规格概要

规格项	配置/说明
Agent名称	QuantAutoTrader
运行平台	OpenClaw（Clawdbot开源版）
管理资金	150,000元（总资产10%）
策略体系	三种策略：抄底反弹、主升浪做多、箱体震荡
阶段识别	四阶段模型 + phase_identifier.py
核心Skills	6个：阶段识别、信号生成、仓位配置、交易执行、风险监控、日志分析
标的配置(V2.0)	抄底/震荡：中证500/1000/2000 | 主升浪：科创50/创业50+主线行业
执行频率	每日15:00决策，次日09:25执行
止损机制	三级：策略止损、账户预警、强制清仓
仓位上限	总80%、单笔33%、单标的40%
人工干预	紧急停止、强制清仓、策略暂停、仓位调整、白名单管理

表11：Agent技术规格概要